

Санкт-Петербургский государственный университет

Кафедра информационно-аналитических систем

Группа 23.Б08-мм

Реализация алгоритма верификации
вероятностных приближённых ограничений
по области в “Desbordante”

СЕНИЧЕНКОВ Пётр Ильич

Отчёт по учебной практике
в форме “Производственное задание”

Научный руководитель:
ассистент кафедры ИАС Чернышев Г. А.

Санкт-Петербург
2026

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Основные определения	5
2.2. Примеры	8
2.3. Алгоритм РАС-Map	11
3. Описание решения	13
4. Эксперименты	18
5. Заключение	22
Список литературы	23

Введение

Профилирование данных — это процесс анализа данных, направленный на извлечение метаданных [6]. Метаданные могут быть как простыми, такими как количество данных, минимальное и максимальное значения, так и сложными, такими как закономерности в данных. Формальные описания закономерностей в данных будем называть *зависимостями* или *примитивами*. Профилирование данных применяется для многих задач, в том числе очистки данных и оптимизации запросов к базам данных [1].

Одним из наиболее известных примитивов является функциональная зависимость (Functional Dependency, FD), когда значения в правой части однозначно определяются значениями в левой части. Для многих примитивов можно также рассмотреть вероятностный и приближённый варианты. Например, вероятностная приближённая зависимость (Probabilistic Functional Dependency, PFD) означает, что большинство значений в правой части однозначно определяются значениями в левой части, а приближённая функциональная зависимость (Approximate Functional Dependency, AFD) означает, что, если значения в левой части мало различаются, то и соответствующие им значения в правой части будут достаточно близки.

Можно обобщить вероятностные и приближённые зависимости, введя вероятностные приближённые ограничения (Probabilistic Approximate Constraints, PAC) [3]. PAC для зависимости D означает, что приближённый вариант D выполняется с некоторой вероятностью. В данной работе рассматриваются вероятностные приближённые ограничения по области (Domain PAC), которые означают, что большинство значений находятся недалеко от некоторой области.

DESBORDANTE — это наукоёмкий профилировщик данных [2] с открытым исходным кодом¹. Данная работа направлена на добавление в проект возможности верификации заданной пользователем Domain PAC.

¹<https://github.com/Desbordante/desbordante-core>

1. Постановка задачи

Целью работы является разработка и внедрение алгоритма верификации Domain PAC в DESBORDANTE. Для достижения цели были поставлены следующие задачи:

1. провести обзор предметной области PAC, включая Domain PAC, Functional Dependency PAC и Unique Key PAC;
2. реализовать алгоритм верификации Domain PAC на C++;
3. разработать модульные тесты для проверки корректности работы алгоритма и соответствующий набор данных для тестов;
4. реализовать Python-привязки к алгоритму верификации;
5. создать примеры использования.

2. Обзор

Вероятностные приближённые ограничения были впервые введены в 2003 году в работе F. Korn и др. [3]. В дальнейшем РАС не изучались [5].

РАС могут применяться для различных задач в случаях, когда данные содержат небольшие неточности вследствие погрешности измерения и редкие значительные отклонения вследствие ошибок ввода и сбора данных. Например, различные данные о компьютерной сети (входящий и исходящий трафик на узле сети, количество и длина пакетов, проходящих по соединению, и т. д.) [3].

2.1. Основные определения

Все определения в данном разделе, кроме опр. 6, взяты из статьи F. Korn и др. [3]. При этом, использованы традиционные для данной области обозначения.

Пусть $R(U)$ — схема отношения (в терминах реляционной модели данных), где R — имя отношения, а $U = \{ A_1, \dots, A_n \}$ — множество атрибутов. Конечный набор кортежей над $R(U)$ будем называть *отношением*. Для каждого атрибута A его область определения обозначать $\text{dom}(A)$. Значение кортежа t на атрибуте A будем обозначать $t[A] \in \text{dom}(A)$.

Для множества атрибутов $X = \{ A_1, \dots, A_n \}$ будем через $\text{dom}(X)$ обозначать множество наборов $(a_1, \dots, a_n)$, где $a_i \in \text{dom}(A_i)$ для $i = 1, \dots, n$. Аналогично, через $t[X]$ обозначим набор $(t[A_1], \dots, t[A_n])$.

Определение 1 (линейно упорядоченная область). Пусть (X, ρ) — метрическое пространство. Множество $D \subset X$ называется *областью*, если оно открыто и связно.

Область D называется *линейно упорядоченной*, если для любых двух элементов $x, y \in D$ выполнено одно из трёх условий:

$$x < y, \quad x = y, \quad x > y$$

◁

Определение 2 (расстояние до множества). Пусть (X, ρ) — метрическое

пространство, $D \subset X$ — множество.

Для точки $x \in X$ расстоянием от x до множества D назовём следующую величину:

$$\rho(x, D) = \inf_{y \in D} \rho(x, y)$$

◁

Определение 3 (Domain PAC). Пусть X — множество атрибутов, на множестве $\text{dom}(X)$ задана метрика ρ , $D \subset \text{dom}(X)$ — линейно упорядоченная область.

Будем говорить, что выполняется *вероятностное приближённое ограничение по области D* (*Domain Probabilistic Approximate Constraint, Domain PAC*), если значения $x \in \text{dom}(X)$ находятся на расстоянии не более ε от D с вероятностью не меньше δ :

$$\Pr(\rho(x, D) \leq \varepsilon) \geq \delta.$$

◁

Определение 4 (FD PAC). Пусть X, Y — множества атрибутов, на множествах $\text{dom}(X), \text{dom}(Y)$ заданы метрики ρ_X, ρ_Y соответственно.

Функциональное вероятностное приближённое ограничение (*Functional Dependency Probabilistic Approximate Constraint, FD PAC*) $X \rightarrow Y$ означает, что, если значения в левой части не сильно различаются (не более, чем на Δ_l), то и соответствующие им значения в правой части будут различаться не более, чем на ε_l с вероятностью не меньше δ , то есть, если

$$\rho_X(t_i[A_l], t_j[A_l]) \leq \Delta_l \quad \forall A_l \in X,$$

то

$$\Pr(\rho_Y(t_i[B_l], t_j[B_l]) \leq \varepsilon_l) \geq \delta \quad \forall B_l \in Y.$$

◁

FD PAC являются обобщением функциональных зависимостей (FD).

Определение 5 (UK PAC). Пусть X — множество атрибутов, на $\text{dom}(X)$ задана метрика ρ .

Вероятностное ограничение целостности по уникальному ключу (*Unique Key Probabilistic Approximate Constraint, UK PAC*) означает, что вероятность, что существует несколько кортежей с близкими значениями ключевых атрибутов (значения различаются не более, чем на ε_l), не превосходит δ_l :

$$\Pr(\rho(t_i[A_l], t_j[A_l]) \leq \varepsilon_l) \leq \delta_l \quad \forall A_l \in X$$

◁

Введём также модифицированное определение UK PAC, лучше подходящее для практического использования.

Определение 6 (UCC PAC). *Вероятностное приближённое ограничение целостности по уникальной комбинации столбцов (Unique Column Combination Probabilistic Approximate Constraints, UCC PAC)* означает, что вероятность, что существует несколько кортежей с близким значением ключа (значения различаются не более, чем на ε), не превосходит δ :

$$\Pr(\rho(t_i[X], t_j[X]) \leq \varepsilon) \leq \delta$$

◁

Таким образом, UK PAC задаёт ограничение для каждого атрибута по-отдельности, а UCC PAC — для комбинации атрибутов. При этом, UK PAC можно выразить через UCC PAC:

$$\text{UK PAC}_{\varepsilon_1, \dots, \varepsilon_n}^{\delta_1, \dots, \delta_n}(\{A_1, \dots, A_n\}) = \text{UCC PAC}_{\varepsilon_1}^{\delta_1}(A_1) \wedge \dots \wedge \text{UCC PAC}_{\varepsilon_n}^{\delta_n}(A_n)$$

Подробное сравнение UK PAC и UCC PAC приведено в разделе 2.2.

UCC PAC является обобщением уникальных комбинаций столбцов (Unique Column Combination, UCC).

Таблица 1: Результаты измерений производимых деталей

Тип	Диаметр, мм	Масса, г
A	44	350
A	39	330
B	90	100
A	48	360
B	79	120

2.2. Примеры

Domain PAC

В табл. 1 приведены результаты измерений деталей, производимых станком: диаметр по одному из измерений и масса, а также, тип детали. Стандарт качества требует, чтобы не менее 90% деталей имели диаметр в пределах 5мм от эталонного диаметра и массу — в пределах 30г от эталонной массы. Эталонные значения диаметра и массы для различных типов деталей приведены в табл. 2.

Таблица 2: Эталонные характеристики для различных типов деталей

Тип	Диаметр, мм	Масса, г
A	45	350
B	85	115

Таким образом, не менее 90% деталей должны иметь диаметр в пределах $(45, 85) \pm 5$ мм. Это можно записать в виде Domain PAC на столбце Диаметр по области $(45, 85)$:

$$\Pr(x \in (45, 85) \pm 5) \geq 0.9$$

Можно ввести более строгое условие: не менее 90% деталей должны иметь диаметр в пределах $(45, 85) \pm 5$ мм и массу в пределах $(115, 350) \pm 30$ г. Это условие можно записать в виде Domain PAC на множестве атрибутов {Диаметр, Масса}, задав ρ как нормированную евклидову

метрику:

$$\rho((x_1, y_1)) = \sqrt{\left(\frac{x_1 - x_2}{85 - 45}\right)^2 + \left(\frac{y_1 - y_2}{350 - 115}\right)^2}$$

Для удобства введём обозначения

$$A_1 = \frac{45}{85 - 45}, \quad B_1 = \frac{85}{85 - 45}, \quad A_2 = \frac{115}{350 - 115}, \quad B_2 = \frac{350}{350 - 115}$$

$$\varepsilon = \sqrt{\left(\frac{5}{85 - 45}\right)^2 + \left(\frac{30}{350 - 115}\right)^2}$$

$$\Pr\left(\rho((x_1, y_1), (A_1, B_1) \times (A_2, B_2)) \leq \varepsilon\right) \geq 0.9$$

FD PAC

Domain PAC, введённые в предыдущем разделе, не очень точно отражают требуемые ограничения на табл. 1. При помощи FD PAC на столбцах {Тип} $\rightarrow$ {Диаметр, Масса} можно ввести более точное ограничение, утверждающее, что не менее 90% деталей одного типа имеют похожие диаметр и массу:

Если $t_i[\text{Тип}] = t_j[\text{Тип}]$, то

$$\Pr\left(\rho((t_i[\text{Диаметр}], t_i[\text{Масса}]), (t_j[\text{Диаметр}], t_j[\text{Масса}])) \leq \varepsilon\right) \geq 0.9$$

Эта FD PAC имеет $\Delta = 1$.

Можно также ввести ограничение, утверждающее, что все детали, имеющие похожий диаметр, должны иметь похожую массу. Это будет FD PAC {Диаметр} $\rightarrow$ {Масса}:

$$|t_i[\text{Диаметр}] - t_j[\text{Диаметр}]| \leq 5 \implies |t_i[\text{Масса}] - t_j[\text{Масса}]| \leq 30$$

Эта FD PAC имеет $\delta = 1$, и является также метрической функциональной зависимостью (Metric FD) [4].

2.2.1. UK PAC и USS PAC

В табл. 3 приведены координаты точек, из которых в течение дня приходили сигналы о повышенной температуре от датчиков,

Таблица 3: Координаты точек, из которых приходили сообщения о повышенной температуре

Широта, м	Долгота, м
44	350
0	25
45	348
1	-3
10	300
10	-2

обнаруживающих лесные пожары. Единичный сигнал может быть проигнорирован, так как датчик мог нагреться на солнце или от костра. Также датчик мог отправить сигнал из-за ошибки оборудования или проблем со связью. Выезд пожарной бригады требуется только если были получены сигналы от нескольких датчиков, расположенных недалеко друг от друга.

Запишем это условие при помощи UK РАС по столбцам Широта и Долгота (будем дальше обозначать Ш и Д соответственно):

$$\Pr(|t_i[\text{Ш}] - t_j[\text{Ш}]| \leq 10) \leq 0.1$$

$$\Pr(|t_i[\text{Д}] - t_j[\text{Д}]| \leq 10) \leq 0.1$$

Однако, данная UK РАС некорректна: например, датчики с координатами $(0, 25)$ и $(1, -3)$ будут учтены как одинаковые в первой части ограничения (по столбцу Широта), хотя они находятся далеко друг от друга. В данном случае ограничение лучше записать в виде USS РАС по комбинации столбцов $\{\text{Широта}, \text{Долгота}\}$. В качестве ρ используем евклидову метрику:

$$\rho((x_1, y_1), (x_2, y_2)) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

$$\Pr(\rho((t_i[\text{Ш}], t_i[\text{Д}]), (t_j[\text{Ш}], t_j[\text{Д}])) \leq 10\sqrt{2}) \leq 0.1$$

2.3. Алгоритм PAC-Man

В работе F. Korn и др. [3] описан алгоритм PAC-Man (PAC Manager), который выполняет две основные задачи для набора PAC:

1. выбирает значения δ^* и ε^* такие, что $\text{PAC}_{\varepsilon^*}^{\delta^*}$ выполняется;
2. отслеживает изменения в данных и сигнализирует о нарушениях PAC.

В данной работе рассматривается только выбор ε^* и δ^* , без проверки стабильности PAC.

Выбор значений δ^* и ε^*

Сначала PAC-Man строит для правил эмпирические функции распределения (Empirical Cumulative Distribution Function, ECDF). Для этого для каждого правила выбираются $\varepsilon_1, \varepsilon_2, \dots, \varepsilon_{99}$ такие, что $\Pr(\varepsilon < \varepsilon_i) \geq \frac{i}{100}$, то есть, эмпирическая (наблюдаемая) вероятность, что ошибка приближения PAC меньше ε_i , не меньше $i\%$.

PAC с большими δ представляют наибольший интерес, так как они означают, что приближённая зависимость почти выполняется. Аналогично, PAC с маленькими ε означают, что вероятностная зависимость выполняется с небольшой погрешностью. Поэтому рассматриваются только $\varepsilon_{\delta_{\min}}, \dots, \varepsilon_{99}$ для некоторого заданного $\delta_{\min}$.

Обычно ECDF содержат “перелом” (“локоть”) в некоторой точке. Выбор $(\varepsilon^*, \delta^*)$ в этой точке будет оптимальным, так как дальнейшее увеличение ε не приведёт к значительному увеличению δ . Для нахождения точки “перелома” ε_s PAC-Man использует эвристический подход, основанный на поиске наиболее различающихся значений ε_i :

$$s = \operatorname{argmax}_{\delta_{\min} < i \leq 99} (\varepsilon_i - \varepsilon_{i-1})$$

Проверка стабильности PAC

Определение 7. Определим расстояние между двумя ECDF $\xi_1 = (\varepsilon_{\delta_{\min}}, \varepsilon_{\delta_{\min}+1}, \dots, \varepsilon_{99})$ и $\xi_2 = (\hat{\varepsilon}_{\delta_{\min}}, \hat{\varepsilon}_{\delta_{\min}+1}, \dots, \hat{\varepsilon}_{99})$ как метрику в простран-

стве L_∞ :

$$d(\xi_1, \xi_2) = \max_{\delta_{min} \leq i \leq 99} |\varepsilon_i - \hat{\varepsilon}_i|$$

◁

Для данной РАС изменения ECDF в течение времени можно записать в виде вектора расстояний $\vec{d}$. Когда данные обновляются, вычисляется расстояние d_{new} между предыдущей и новой ECDF. РАС-Man сравнивает d_{new} со значением $d_{exp} = \text{avg}(\vec{d}) + k \cdot \text{std}(\vec{d})$, где $\text{avg}(\vec{d})$ — среднее значение $\vec{d}$, $\text{std}(\vec{d})$ — среднеквадратичное отклонение $\vec{d}$, и k — заданный параметр. Если d_{new} превышает d_{exp} , то такая РАС нестабильна. В таком случае РАС-Man сигнализирует о нарушении РАС.

3. Описание решения

Предлагаемый алгоритм использует РАС-Map для верификации РАС всех типов. При этом, для каждого типа РАС необходимо вычислить “эмпирические вероятности” — пары (ε, δ) такие, что выполняется $\text{РАС}_\varepsilon^\delta$.

Общими параметрами для всех верификаторов РАС являются $\delta_{\min}$ — минимальное значение δ и n_δ — количество рассматриваемых значений δ . Для верификации РАС каждого типа также задаются другие параметры, например, такие как упорядоченная область и метрика на множестве значений в таблице.

Также в DESBORDANTE алгоритмы верификации вычисляют исключения — наборы кортежей, из-за которых зависимость не выполняется или выполняется с “худшими” параметрами.

Для Domain РАС на множестве атрибутов X по области D каждому кортежу t сопоставим значение $e = \rho(t[X], D)$. Исключение с параметрами ε_1 и ε_2 представляет собой набор кортежей, для которых $\varepsilon_1 < e \leq \varepsilon_2$.

Для FD РАС $X \rightarrow Y$ или USS РАС на множестве атрибутов Y каждой паре кортежей (t_1, t_2) поставим в соответствие значение $e = \rho(t_1[X], t_2[X])$. Исключение с параметрами ε_1 и ε_2 представляет собой набор пар кортежей, для которых $\varepsilon_1 < e \leq \varepsilon_2$. В случае FD РАС рассматриваются только те пары, для которых $\rho(t_1[A_i], t_2[A_i]) \leq \Delta_i$ для всех $A_i \in X$.

Таким образом, исключение не зависит от параметров РАС ε и δ .



Рис. 1: Исключения для Domain РАС на области $(50, 75)$ с $\varepsilon_1 = 3$ и $\varepsilon_2 = 7$

Рассмотрим Domain РАС на столбце Диаметр в табл. 1 по области $(50, 75)$. Значения, которые попадут в исключение с параметрами $\varepsilon_1 = 3$ и $\varepsilon_2 = 7$, показаны на рис. 1. Зелёным цветом отмечена область $(50, 75)$, синим — промежутки, содержащие исключения.

Вычисление “эмпирических вероятностей”

Для вычисления “эмпирических вероятностей” для Domain PAC используется функция `FindEpsilon`, которая вычисляет ε_i для каждой δ_i из интервала $[\delta_{min}, 1]$, а также “уточняет” δ_i : вычисляет максимальное δ'_i такое, что выполняется $PAC^{\delta'_i}_{\varepsilon_i}$.

При чтении таблицы кортежи сортируются по возрастанию в соответствии с заданным на $\text{dom}(X)$ отношением порядка. Это позволяет быстро вычислять ε_i и δ_i^{ref} , в том числе для различных значений параметров δ_{min} и n_δ . Также это позволяет вычислять исключения для любых ε_1 и ε_2 при помощи бинарного поиска.

Выбор домена

Согласно опр. 3, для верификации Domain PAC на множестве атрибутов X необходимо выбрать упорядоченную область D (для этого необходимо выбрать открытое связное множество и задать на нём отношение линейного порядка) и задать на $\text{dom}(X)$ метрику ρ . В предлагаемом алгоритме верификации Domain PAC используются другие параметры, более удобные для пользователя.

Все значения в таблице упорядочиваются по расстоянию от области, то есть

$$x < y \iff d(x, D) < d(y, D), \quad x = y \iff d(x, D) = d(y, D).$$

В результате, для верификации Domain PAC, пользователю не нужно задавать отношение порядка.

Поскольку определение Domain PAC и используемые методы верификации никак не используют открытость множества D , вместо упорядоченной области можно взять любое упорядоченное связное множество. Таким образом, для верификации Domain PAC по множеству атрибутов X достаточно задать связное множество D и функцию расстояния до домена $d(x) = \rho(x, D)$. Для краткости пару (D, d) будем называть *доменом* для данной Domain PAC.

Поддержка произвольных доменов реализована с помощью класса `Domain`. Чтобы воспользоваться верификацией РАС, пользователю необходимо создать подкласс класса `Domain`, реализующий метод `DistFromDomain`, который для заданного значения $t[X]$ вычисляет расстояние от домена до этого значения.

Также определены классы некоторых доменов:

- n -мерный замкнутый шар, который определяется центром c и радиусом r :

$$D = \{ x \in \text{dom}(X) \mid \rho(x, c) \leq r \}.$$

- Замкнутый n -мерный параллелепипед, который определяется двумя углами l и u :

$$\begin{aligned} P = [l, u] &= [l_1, u_1] \times [l_2, u_2] \times \cdots \times [l_n, u_n] = \\ &= \{ x \in \text{dom}(X) \mid l_1 \leq x_1 \leq u_1, l_2 \leq x_2 \leq u_2, \dots, l_n \leq x_n \leq u_n \}. \end{aligned}$$

- Пользовательский “нетипизированный” домен, для которого функция `DistFromDomain` принимает строковое представление значения $t[X]$. Этот класс предоставляет удобный интерфейс для задания простых доменов, для которых не важен (или заранее известен) изначальный тип значения. Кроме того, “нетипизированный” домен необходим для реализации привязок к Python.

Вычисление исключений

Для вычисления исключений между ε_1 и ε_2 при помощи бинарного поиска находятся первое (ближайшее к домену) значение, для которого не выполняется `Domain РАС $\varepsilon_1$` , и первое значение, для которого не выполняется `Domain РАС $\varepsilon_2$` . Кортежи, находящиеся между ними, образуют исключение.

Интеграция алгоритма в DESBORDANTE

Диаграмма классов, участвующих в верификации PAC, показана на рис. 2. Зелёным цветом показаны классы, ранее присутствовавшие в DESBORDANTE, синим — реализованные в ходе данной работы.

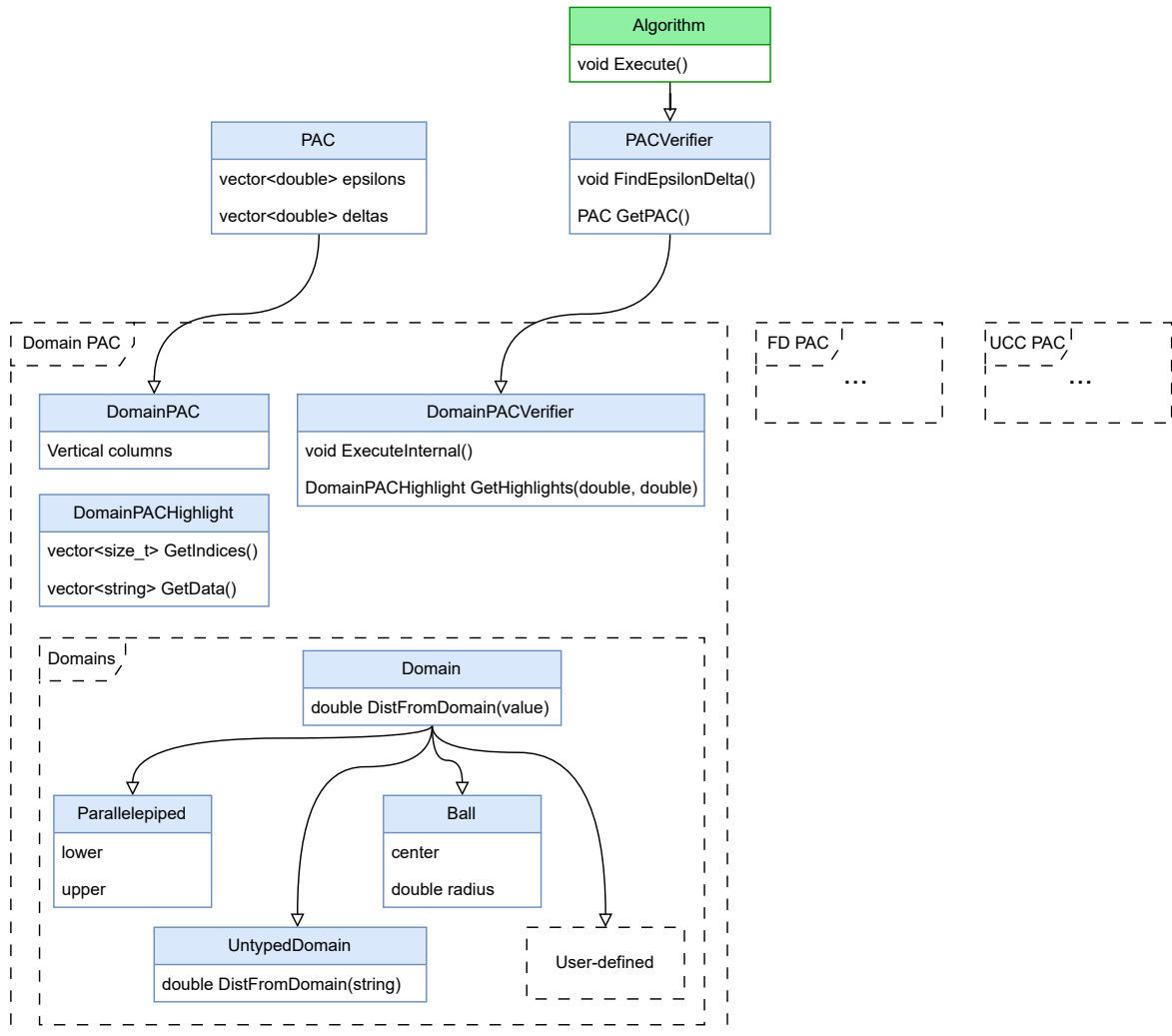


Рис. 2: Диаграмма классов верификации PAC

В DESBORDANTE каждый алгоритм представляется классом, унаследованным от **Algorithm**. Алгоритм PAC-Map реализован в классе **PACVerifier**. От него наследуются классы, представляющие алгоритмы верификации для конкретных типов PAC.

Домены представлены интерфейсом **Domain**. Пользователь может воспользоваться реализованными классами доменов (**Ball** и **Parallelepiped**), определить класс, реализующий интерфейс **Domain** (при использовании

DESBORDANTE в качестве C++-библиотеки), или определить домен при помощи класса `UntypedDomain` (при использовании Python-привязок).

4. Эксперименты

Основная задача эксперимента — оценить масштабируемость алгоритма с увеличением числа строк в таблице и количества атрибутов, участвующих в Domain PAC. Также изучается, как на время работы алгоритма влияют такие факторы, как используемый домен, типы данных в столбцах и общее число атрибутов в таблице.

Таким образом, были поставлены три исследовательских вопроса:

- RQ1.** Как время работы алгоритма зависит от используемого домена, типа данных в таблице и количества атрибутов, участвующих в верификации (“арности” Domain PAC)?
- RQ2.** Как время работы алгоритма зависит от количества строк и столбцов в таблице?
- RQ3.** Каково соотношение времени работы различных стадий алгоритма?

В **RQ1** и **RQ2** учитывается только время работы основного алгоритма без вычисления исключений.

Характеристики системы, на которой проводились измерения:

- операционная система: OpenSUSE Linux 6.18.1-1;
- процессор: Intel Core i7-12700H (20) @ 4.600GHz
- оперативная память: 32 GiB.

Эксперименты проводились на таблицах iowa и EpicVitals² различных размеров. Характеристики тестовых наборов данных приведены в табл. 4.

Каждый эксперимент проводился 30 раз, после чего вычислялись среднее время выполнения и среднеквадратичное отклонение σ .

²Все наборы данных доступны по адресу <https://github.com/Desbordante/desbordante-data/blob/main/datasets.zip>

Таблица 4: Характеристики тестовых наборов данных

Название	Количество строк	Количество столбцов	Размер, МБ
iowa4kk	4 000 000	24	838.2
iowa2kk	2 000 000	24	419.1
iowa1kk	1 000 000	24	209.5
iowa650k	650 000	24	136.2
iowa550k	550 000	24	115.3
EpicVitals4kk	4 000 000	7	104.5
EpicVitals2kk	2 000 000	7	52.3
EpicVitals1kk	1 000 000	7	26.1
EpicVitals650k	650 000	7	16.9
EpicVitals550k	550 000	7	14.3

RQ1

Для ответа на RQ1 были проведены эксперименты на таблице iowa1kk. Эксперименты проводились на доменах Parallelepiped и Ball, для Domain PAC различных “арностей”. При этом, использовались столбцы, содержащие различные типы данных. Результаты измерений приведены в табл. 5.

Можно видеть, что время работы алгоритма очень слабо зависит от “арности” Domain PAC и типа данных и совсем не зависит от выбора домена.

RQ2

Для ответа на RQ2 были проведены эксперименты на таблицах, содержащих различное количество строк и столбцов. Во всех экспериментах использовался домен Ball, “арность” Domain PAC равна единице. Все столбцы, по которым проводилась верификация, содержат только целые числа. В табл. 6 приведены среднее время работы алгоритма и

Таблица 5: Среднее время работы алгоритма и среднеквадратичное отклонение для различных доменов и на различном количестве атрибутов

Домен	Количество атрибутов	Тип данных	Время, с	σ , с
Ball	1	целые числа	12.12	0.16
Parallelepiped	1	целые числа	11.95	0.14
Ball	1	даты	11.97	0.11
Parallelepiped	1	даты	12.06	0.12
Ball	1	строки	12.17	0.16
Parallelepiped	1	строки	12.23	0.22
Ball	4	строки	12.97	0.06
Parallelepiped	4	строки	13.06	0.20
Ball	4	смешанный	12.10	0.23
Parallelepiped	4	смешанный	12.31	0.11
Ball	19	смешанный	13.84	0.16
Parallelepiped	19	смешанный	13.79	0.13

среднеквадратичное отклонение.

Из табл. 6 видно, что время работы алгоритма сильно зависит от размеров таблицы, на которой запускается верификация. Это происходит из-за того, что перед началом работы алгоритма требуется прочитать весь набор данных.

RQ3

В табл. 7 приведены среднее время работы и среднеквадратичное отклонение для различных стадий работы алгоритма на таблице iowa650k. Видно, что чтение набора данных (загрузка таблицы) требует гораздо больше времени, чем остальные стадии.

Также следует отметить, что во всех таблицах приведено время, затраченное на подготовку данных. Время работы самой процедуры верификации Domain PAC зависит только от n_δ и пренебрежимо мало

Таблица 6: Среднее время работы алгоритма и среднеквадратичное отклонение на различных наборах данных

Количество строк	Количество столбцов	Время работы, с	σ , с
4 000 000	24	46.70	0.35
2 000 000	24	23.46	0.39
1 000 000	24	12.12	0.16
650 000	24	8.15	0.08
550 000	24	6.77	0.11
4 000 000	7	9.37	0.10
2 000 000	7	4.73	0.07
1 000 000	7	2.41	0.04
650 000	7	1.59	0.02
550 000	7	1.37	0.03

Таблица 7: Время работы различных стадий алгоритма на наборе данных iowa650k

Стадия алгоритма	Время, мс	σ , мс
Загрузка таблицы	7878.3	217.5
Объединение значений в кортежи	7.9	0.5
Вычисление расстояний до домена	3.0	0.1
Сортировка кортежей	23.6	0.8

для любых разумных значений n_δ (меньше 1мс для $n_\delta = 10^9$). Аналогично, время, необходимое для вычисления исключений зависит только от числа строк в таблице и составляет меньше 3мс на таблице, состоящей из 4 000 000 строк.

Такое разбиение алгоритма на подготовку данных (**LoadData**) и верификацию (**Execute**) используется во многих алгоритмах в DESBORDANTE и позволяет эффективно верифицировать Domain PAC с различными параметрами δ_{min} и n_δ , а также вычислять исключения для различных ε_1 и ε_2 на одной таблице, не подготавливая данные каждый раз заново.

5. Заключение

В ходе данной работы в проект была добавлена поддержка верификации вероятностных приближённых ограничений по области. Результаты работы:

1. проведён обзор предметной области;
2. реализован алгоритм верификации Domain PAC;
3. разработаны модульные тесты и соответствующий набор данных для тестов;
4. реализованы Python-привязки к алгоритму верификации;
5. созданы примеры использования.

Исходный код доступен на GitHub³. Изменения находятся на стадии рассмотрения.

³<https://github.com/Desbordante/desbordante-core/pull/611>

Список литературы

- [1] [Data Profiling](#) / Ziawasch Abedjan, Lukasz Golab, Felix Naumann, Thorsten Papenbrock. — Springer Cham, 2018. — Nov. — ISBN: [978-3-031-00737-8](#).
- [2] [Desbordante: from benchmarking suite to high-performance science-intensive data profiler](#) / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: <https://doi.org/10.1145/3703323.3703725>.
- [3] Korn Flip, Muthukrishnan S., Zhu Yunyue. Checks and balances: Monitoring data quality problems in network traffic databases // Proceedings 2003 VLDB Conference / Elsevier. — 2003. — P. 536–547.
- [4] [Metric Functional Dependencies](#) / Nick Koudas, Avishek Saha, Srivastava Divesh, Suresh Venkatasubramanian // 2009 IEEE 25th International Conference on Data Engineering. — 2009. — P. 1275–1278.
- [5] Song Shaoxu, Chen Lei. [Integrity Constraints on Rich Data Types](#). — Springer, 2023. — Mar. — ISBN: [978-3-031-27176-2](#).
- [6] Ziawasc Abedjan, Lukasz Golab, Felix Naumann. Profiling Relational Data: A Survey // [The VLDB Journal](#). — 2015. — Aug. — Vol. 24, no. 4. — P. 557–581.